**Public editions adapted from real Career Agent Pro outputs**

Edição pública adaptada de uma saída real do produto. Material educacional gerado por IA e revisado para publicação. Todos os contextos de arquitetura, equipes, incidentes, requisitos e avaliação são hipóteses didáticas de plataformas fictícias. Não descrevem informações internas confirmadas, questões oficiais, contratação ou estratégia de empresas reais. Sem vínculo ou endosso de Prezensa, Globo, WhatsApp ou Meta.

# Análise de oportunidade Android em streaming

Engenharia Android para plataforma fictícia de streaming. Nível didático: sênior. Este documento não descreve uma vaga real.

> Premissa didática: uma plataforma fictícia de streaming precisa manter reprodução estável, conteúdo offline e evolução incremental de uma base Android. A profundidade técnica é escolhida para este exercício; não corresponde a critérios reais de contratação.

## Resumo Executivo

Este cenário explora a atuação de engenharia Android em um produto fictício de vídeo e notícias. O objetivo é conectar ciclo de vida, reatividade, persistência e qualidade de reprodução a decisões justificáveis.

A combinação de requisitos exige raciocinar sobre falhas, limites de responsabilidade e migração sem interrupção. O material propõe caminhos de estudo; não prevê aprovação nem senioridade oculta de uma vaga.

## Desafios de produto e engenharia

Mapeie hipóteses de produto e traduza-as em requisitos verificáveis, sem presumir necessidades internas de uma empresa real.

### Escalabilidade e estabilidade em apps de streaming de massa

Considere um serviço fictício de streaming de grande escala. Bufferização, falhas de rede e consumo de memória afetam a experiência; instrumente tempo até o primeiro frame, rebuffering e falhas por sessão antes de priorizar otimizações.

Implicação técnica: estudar ciclo de vida Android, memória, threading e reatividade; estes temas são adotados como requisitos de preparação, sem alegação sobre uma descrição real de vaga.

### Manutenção e evolução de base de código legada (Java → Kotlin)

Hipótese de estudo: a base fictícia combina Java e Kotlin. Avalie interoperabilidade, nulabilidade, cancelamento e fronteiras de módulos antes de escolher uma estratégia gradual de migração.

**Implicação técnica:** capacidade de transitar entre paradigmas, entender interoperabilidade Java/Kotlin e tomar decisões de refatoração com segurança.

---

### Autonomia e entrega em ambiente 100% remoto

Premissa organizacional: o exercício supõe colaboração remota e revisão de arquitetura existente. Discuta onboarding técnico, documentação de decisões e entregas incrementais; não há contrato real de trabalho associado.

**Implicação técnica:** familiaridade sólida com MVVM, Clean Architecture, injeção de dependência, Git flow e capacidade de navegar uma codebase existente com agilidade.

---

## Stack Tecnológica

### Competências centrais

| Tecnologia | Contexto |
|---|---|
| **Kotlin** | Linguagem principal de desenvolvimento Android |
| **Java** | Presente como legado - conhecimento esperado |
| **RxJava** | Escolhido como tópico de estudo para programação reativa |
| **LiveData** | Componente de arquitetura reativa do Jetpack |
| **Kotlin Flow / Coroutines** | Paradigma moderno reativo - escolhido para estudar fluxos reativos |
| **Android Lifecycle** | Exigido com nível avançado |
| **Injeção de Dependência** | Escolha do exercício - framework a escolher e justificar |
| **Gradle** | Gerenciamento de dependências - adotado como tópico de estudo |
| **Git** | Versionamento - adotado como objetivo didático |

---

### Tópicos complementares escolhidos

| Tecnologia | Raciocínio |
|---|---|
| **Hilt / Dagger 2** | Frameworks de DI dominantes no ecossistema Android corporativo |
| **Maven** | Alternativa de tooling a comparar quando pertinente |
| **ViewModel / Room / Navigation** | Componentes Jetpack associados ao ciclo de vida Android |

| Tecnologia | Raciocínio |
|---|---|
| **Retrofit / OkHttp** | Padrão de mercado para chamadas de rede |
| **JUnit / Espresso / Mockito** | Objetivo: testar estado, cancelamento e efeitos de falhas sem depender de rede real |

### Tópicos de aprofundamento

| Tecnologia / Conhecimento | Contribuição para o exercício |
|---|---|
| **Kotlin Coroutines avançado** | Flow é adotado para comparar cancelamento, estado e eventos |
| **ExoPlayer / Media3** | Estudar player de mídia e seus lifetimes é parte do escopo didático |
| **Clean Architecture + MVVM ou MVI** | Padrão em produtos Android de grande escala |
| **Jetpack Compose** | Alternativa de UI para comparação: Compose, considerando interoperabilidade e custo de migração |
| **CI/CD para Android** (Bitrise, GitHub Actions, Fastlane) | Times grandes dependem de automação de build e deploy |
| **Testes unitários e instrumentados** | Praticar testes e justificar limites de cobertura |
| **DRM / streaming de vídeo** | Altamente relevante para o contexto da plataforma fictícia de streaming |

## Profundidade e escopo do exercício

| Dimensão | Avaliação |
|---|---|
| Conhecimentos de entrada | Pleno |
| **Profundidade escolhida** | **Pleno-Sênior / Sênior** |
| Complexidade técnica esperada | Alta |

A profundidade sênior deste exercício decorre das decisões propostas, não de uma interpretação de requisitos empresariais. Discuta:

- Exige **conhecimento avançado** de Android Lifecycle - não básico ou intermediário
- A combinação **RxJava + LiveData + Flow** indica convivência com múltiplos paradigmas reativos, típica de codebase madura e complexa
- O cenário exige justificar limites entre reprodução, catálogo e persistência
- Use uma simulação estruturada, com requisitos, tempo escolhido e revisão das evidências

> Use a rubrica para identificar lacunas de estudo; nenhum nível ou resultado de seleção real é inferido.

# Premissas de organização do cenário

Premissa organizacional: colaboração entre módulos e tomada de decisão orientada por métricas, escolhidas para ampliar a complexidade do exercício.

Premissa fictícia: várias pessoas mantêm componentes de diferentes gerações. RxJava, LiveData e Flow podem coexistir. Planeje contratos, adapters e testes de regressão para uma evolução gradual.

**O que isso significa na prática:**

- Pratique registrar hipóteses, decisões e evidências para revisão assíncrona
- Defina documentação mínima de contratos, ownership e invariantes no protótipo
- Velocidade importa, mas **qualidade e consistência são igualmente valorizadas**
- Pergunta de diligência, fora da ficção: quais etapas e responsáveis existem em uma oportunidade real? Não se presume dupla aprovação.

---

# Roteiro de preparação e autoavaliação

> Esta seção é uma rubrica independente: pratique profundidade técnica, comunicação e comparação de alternativas. Não descreve entrevistadores ou cultura de uma empresa real.

## O que tende a ser valorizado

- **Ownership técnico** - saber *por que* cada decisão arquitetural foi tomada, não apenas seguir padrões
- **Domínio de reatividade** - entender diferenças e trade-offs entre RxJava, LiveData e Flow
- **Ciclo de vida Android profundo** - experiência real com vazamentos de memória, race conditions e comportamento em background
- Testes: propor cobertura para cancelamento, falhas e transições de estado, justificando limites
- **Comunicação técnica clara** - times distribuídos dependem de quem documenta, explica e alinha por escrito
- **Pragmatismo** - saber quando refatorar e, principalmente, quando não refatorar

## Lacunas a explorar na autoavaliação

- Não saber diferenciar **LiveData vs StateFlow vs SharedFlow**
- Confundir o ciclo de vida de **Activity vs Fragment vs ViewModel**
- Ter usado injeção de dependência apenas em tutoriais - sem experiência real em projetos
- Responder “uso o que o time decidir” sobre arquitetura sem demonstrar conhecimento próprio
- Não saber o que acontece quando o app vai para background durante uma operação assíncrona
- Apresentar código sem testes **e** sem justificativa para isso

> Na simulação, compare poucas decisões em profundidade: requisitos, invariantes, alternativas e mecanismos de observação. As perguntas abertas abaixo servem para praticar raciocínio, sem previsão de conteúdo de seleção.

---

# Trilha de aprofundamento

## Técnico - Fundamentos

- Diferenças entre **RxJava, LiveData e Kotlin Flow** - quando usar cada um e por quê
- **Coroutines:** structured concurrency, escopos, dispatchers, cancelamento e tratamento de exceções
- **Android Lifecycle:** comportamento em rotação de tela, app em background e kill de processo
- **ViewModel:** por que sobrevive à rotação e como funciona internamente
- **Injeção de Dependência:** diferença entre Hilt e Dagger, funcionamento de um grafo de dependências
- **Gradle:** diferença entre `implementation`, `api` e `compileOnly`; build variants; product flavors

## Arquitetura

- **Clean Architecture** no Android: separação de camadas, casos de uso, repositórios
- **MVVM vs MVI:** trade-offs e quando usar cada abordagem
- **Modularização:** por que modularizar, tipos de módulo (feature, core, data)
- **Gerenciamento de estado:** consistência de UI em fluxos assíncronos complexos
- **Offline-first:** estratégias de cache, Room e sincronização

## Avançado - Diferenciais

- **Memory leaks:** como identificar, ferramentas (LeakCanary), causas mais comuns
- **Threading avançado:** diferença entre dispatchers `Main`, `IO` e `Default` em Coroutines
- **Testes:** como mockar dependências com Hilt em testes instrumentados; unit test vs integration test
- **Performance de listas:** RecyclerView, DiffUtil, Paging 3
- **Streaming de mídia:** ExoPlayer, Media3, DRM - especialmente relevante para o contexto da plataforma fictícia de streaming

## Comunicação e colaboração: perguntas de preparação

- Como você lida com código legado que não pode ser reescrito agora?
- Descreva uma situação em que você tomou ownership de um problema que não era formalmente seu.
- Como você equilibra velocidade de entrega e qualidade técnica?
- Você já trabalhou em time distribuído remotamente? Como foi a comunicação?
- Como você se mantém atualizado tecnicamente?

# Riscos e Mitigações

| Risco | Descrição | Como mitigar |
|---|---|---|
| Estado e lifetimes | Confundir estado de tela com recursos de reprodução | Definir ownership; testar rotação, background e recriação do processo |

| Risco | Descrição | Como mitigar |
|---|---|---|
| Migração reativa | Misturar RxJava e Flow sem política de cancelamento | Mapear adapters e testar cancelamento, erros e ausência de dupla assinatura |
| Cache e frescor | Exibir conteúdo antigo como atualizado | Modelar origem, idade, atualização em andamento e falha parcial |
| Mudança de conta | Reutilizar dados e coletores do perfil anterior | Isolar cache, cancelar trabalhos e testar logout/login concorrente |
| Performance | Otimizar sem medir impacto no dispositivo | Instrumentar startup, frames, memória e rebuffering com cargas sintéticas |
| Escopo da oportunidade real | Deduzir contrato ou etapas de seleção pelo domínio técnico | Confirmar condições com a fonte responsável; não são premissas desta amostra |

## Objetivos de aprendizagem

| Oportunidade | Como aproveitar |
|---|---|
| Reprodução robusta | Construir uma máquina de estados e demonstrar recuperação após interrupções |
| Arquitetura evolutiva | Comparar migração por fronteiras, adapters e substituição completa; registrar custo e reversibilidade |
| Qualidade observável | Relacionar testes de contrato e de estado com métricas de experiência |
| Colaboração técnica | Documentar invariantes e limites do módulo para permitir trabalho paralelo |
| Investigação de produto | Observar o protótipo com rede/contas diferentes, sem inferir detalhes internos de apps de terceiros |

> Síntese do estudo: implemente uma fatia de player e feed, simule rotação, perda de conexão e recriação do processo. Registre decisões, métricas e limitações em vez de atribuir comportamentos a produtos reais.

## Limites e evidências da implementação

Separe estado de UI, recursos do player e dados persistidos. ViewModel ajuda em mudanças de configuração, mas não substitui restauração após morte do processo. Serviços também podem ser encerrados; definir recuperação e persistir apenas o necessário. No feed, diferencie cache vazio, dado antigo, atualização em andamento e falha de atualização.

Compare arquiteturas por invariantes e evidências: testes com scheduler controlado, cancelamento, callbacks após destruição da view, alteração de conta, relógio incorreto e falhas parciais. Metas numéricas são parâmetros do exercício; medir em uma matriz sintética de dispositivos e cargas antes de afirmar que uma alternativa cumpre o orçamento.

# Referências técnicas e escopo

Referências de APIs públicas para estudo; não são evidência sobre arquitetura ou processos de empresas reais. Os contratos Kotlin são exemplos educacionais, sem implementação de produtos privados.

- Android: coroutines e ciclo de vida
- Android: WorkRequests e constraints
- Media3: ExoPlayer e application thread

**Career Agent PRO**

Edição pública adaptada de saída real. Profundidade técnica, hipóteses explícitas e decisões verificáveis.

www.career-agent-pro.com

support@main.career-agent-pro.com

© 2026 Career Agent PRO. Todos os direitos reservados.

ADAPTED PUBLIC EDITION